

LLM 预训练之 Tokenizer

Tokenization • 深度研究手册

从“文本切分”到“模型离散接口”：BPE、Unigram、SentencePiece、Byte-level、Vocabulary Scaling、多语言 Token Tax 与 Tokenizer Expansion

核心视角：Tokenizer 同时决定序列长度 L 、词表大小 V 、有效上下文、训练/推理成本、语言公平性与协议兼容性

Research cut-off: 2026-08-13 | 01-B-Agent

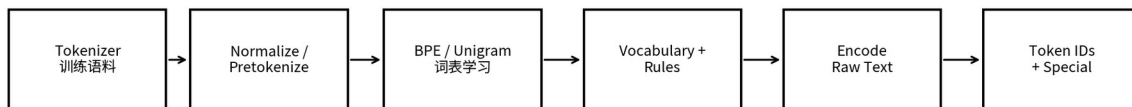
执行摘要：Tokenizer 是“模型架构的一部分”，不是无关紧要的预处理

现代 LLM 常把 Tokenizer 固定在预训练之前，因此它看起来像一个静态工具；但从优化和系统角度，Tokenizer 实际上定义了模型学习世界的离散坐标系。相同原始文本经过不同 Tokenizer，会得到不同的序列长度、不同的预测目标、不同的词表容量需求和不同的计算路径。

核心判断	为什么成立	工程后果
Tokenizer 改变优化问题	$\tau(x)$ 决定 next-token targets 和序列长度	不同 tokenizer 的 token-level loss/PPL 不可直接横比
压缩率会改变有效上下文	同一 128K token window 能容纳的字符/代码量不同	Context Length 必须同时报告 chars/token 或 bytes/token
词表大小参与 scaling	V 增大可缩短序列，但 Embedding/LM Head 变大	存在 compute/model/domain 相关 optimum，而非固定 32K
Tokenizer 自带数据偏好	BPE merge 频率来自 tokenizer 训练语料	Tokenizer corpus 需要独立 mixture、版本和审计
多语言会产生 Token Tax	低覆盖语言被切得更碎	成本、延迟和有效上下文可能系统性不公平
后续改 tokenizer 很昂贵	Embedding 与 LM Head 都绑定 token IDs	最好预留协议 token；扩词表需专门初始化与 CPT

最重要的 Know-why 预训练不是在字符空间学习 $P(\text{text})$ ，而是在 Tokenizer 定义的离散序列空间学习 $P(z_1, \dots, z_n)$ 。Tokenizer 因此改变“一个训练 token 到底代表多少原始信息”，也改变每个 FLOP 买到多少字符、多少代码和多少跨语言知识。

Tokenizer 不是一个函数，而是一套“训练数据 -> 离散符号系统 -> 模型接口”的设计



Tokenizer 的输出同时决定：序列长度 L、输出字典大小 V、特殊协议、有效上下文与每字符计算成本

图 1 Tokenizer 系统边界：词表训练和在线编码是两条不同但耦合的链路。

0. 数学对象：Tokenizer 改变的是训练序列，而不只是文本表示

给定原始字符串 x ，Tokenizer τ 将其映射为可逆或近似可逆的 token 序列 $z = \tau(x) = (z_1, \dots, z_n)$ ，其中 $z_i \in \{1, \dots, V\}$ 。Decoder-only LM 优化：

$$L_{\tau}(x) = - \sum_i \log P_{\theta}(z_i | z_{<i})$$

注意 n 和 V 都由 τ 决定。即使底层 Transformer 完全相同，换 Tokenizer 后，loss 的求和项数量、每一步分类空间、每段 raw text 的 attention 长度都会改变。

量	由谁决定	对训练的直接影响
n = token count	Tokenizer 粒度 / 语料覆盖	Attention/MLP 执行次数、packing、有效 batch 原文量
V = vocab size	词表预算与算法	Embedding 参数、LM Head 输出维度、softmax/通信成本
token boundaries	Pretokenizer + merge/unigram rules	形态、数字、代码、空白和符号的可学习结构
special token IDs	协议设计	文档边界、BOS/EOS、FIM、chat/tool/multimodal 控制
normalization	Unicode/whitespace policy	是否 lossless、是否把不同字符串折叠为同一表示

比较模型时的坑 Token-level perplexity 依赖词表和分词粒度，因此跨 tokenizer 直接比较 PPL 会误导。更稳妥的跨 tokenizer 语言建模指标是 bits-per-byte / bits-per-character，前提是规范化和解码定义清楚。

1. 为什么从 Word 走向 Subword、Byte 与 Token-free

粒度	优点	核心问题	适用理解
Word	语义直观、序列短	开放词表、词形爆炸、跨语言困难	早期 NLP
Character	无 OOV、语言规则少	序列长；语义单元过细	形态/拼写敏感任务
Byte	固定 256 基础字典、任何 UTF-8 可表示	序列更长；Unicode 字符可跨多个 byte	ByT5 / byte fallback / byte-BPE
Subword	兼顾覆盖和压缩	边界是统计结果，不一定符合形态	现代 LLM 主流
Dynamic / Token-free	减少固定词表技术债	训练与推理架构更复杂	研究前沿

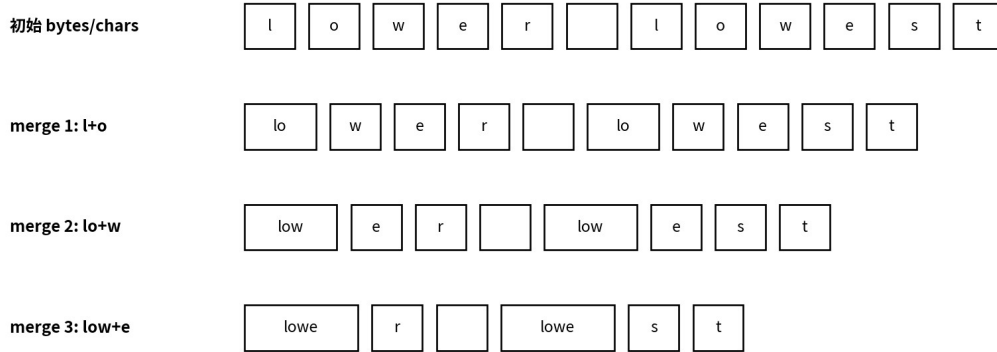
Sennrich 等在 2016 年将 BPE 风格 subword 用于 NMT 稀有词问题，核心是让频繁词或词片段保持较长 token，同时把稀有词拆成可复用单元。[1]

ByT5 则展示了另一条路径：直接在 UTF-8 bytes 上训练标准 Transformer，虽然序列更长，但避免复杂 tokenizer，并在噪声、拼写和发音敏感任务上表现出更强鲁棒性。[9]

2. BPE：为什么它简单、可扩展，并成为现代 LLM 主流之一

经典 BPE 的训练过程可以理解为“频率驱动的字典压缩”：先从字符或 bytes 作为基本符号开始，统计相邻 pair，反复把最常见 pair 合并成新 token，直到达到目标 merge 数或词表规模。

BPE 的核心直觉：把频繁共现的短单元逐步压缩成更长 token（玩具示意）



真实 BPE 从整个训练语料的频率统计中反复选择高频相邻对；byte-level BPE 以 256 个 bytes 保证开放词表覆盖。

图 2 BPE 玩具示意。真实训练是在整个 corpus 上选择频繁 pair，而不是针对单个单词。

环节	BPE Know-how	为什么重要
Base alphabet	Unicode chars 或 256 bytes	byte base 可以彻底消除未知字符
Pretokenization	先用 regex/whitespace 等约束 merge 边界	会显著改变 code、数字、标点和多语言压缩
Pair counting	统计全语料相邻 pair	Tokenizer 数据 mixture 会直接进入 merge ranking
Merge order	保存 pair -> new token 的优先级	编码阶段必须严格复现，否则 ID 不兼容
Vocab budget	base + merges + special tokens	决定压缩与 embedding/head 成本

OpenAI 的 tiktoken 教学实现清楚展示了现代 byte-level BPE 的两步：先用 Unicode regex 做 approximate-word split，再把每个 piece 转成 UTF-8 bytes 并按 merge ranks 合并。[5]

BPE 的局限 BPE 的“高频 pair = 好 token”是压缩启发式，不等价于“对语言建模最优”。Bostrom & Durrett 的控制实验发现 Unigram LM 在英语和日语的 masked LM 预训练中可匹配或超过 BPE，并更贴近形态结构。

3. Unigram LM、WordPiece、SentencePiece：不要把“算法”和“工具框架”混为一谈

名称	本质	训练方式	分词方式	关键能力
BPE	merge-based subword algorithm	从小词表逐步合并	按 learned merge priority	简单、高速、主流
Unigram LM	probabilistic subword model	从大候选集经 EM/剪枝缩小	Viterbi / 可采样多个 segmentation	可做 subword regularization
WordPiece	likelihood-oriented subword family	与 BPE 相似但 scoring 不同	常结合语言相关 pretokenization	BERT 系经典
SentencePiece	Tokenizer framework	可训练 BPE/Unigram/char/word	从 raw sentence，显式处理空白	语言独立、normalization、byte fallback

SentencePiece 的关键创新不是“发明 BPE”，而是把 raw sentence、Unicode normalization、whitespace 表示、词表训练与 ID 映射做成语言无关的一体化系统，并支持 BPE 与 Unigram 两种主要算法。[3][4]

Kudo 的 Subword Regularization 利用同一句话存在多种可行分词这一事实，在训练时概率采样不同 segmentation；该工作也提出基于 Unigram LM 的新分词算法，并报告低资源与域外设置更明显的收益。[2]

SentencePiece 参数	工程含义	常见决策
character_coverage	哪些 Unicode 字符必须进入基础字母表	小字母表语言可 1.0；CJK 常需处理长尾噪声
split_by_unicode_script	是否允许 token 跨 script	多语言常建议阻止奇怪跨 script merge
split_by_number / split_digits	数字与文本边界 / 是否逐位切分	数学、金融、数字推理需单独 ablation
split_by_whitespace	token 是否跨空白	影响 code、自然语言与可逆性
byte_fallback	未知字符转 UTF-8 byte tokens	现代 LLM 通常应避免 <unk>
normalization_rule	NFKC / identity 等	决定“原字符串”是否被折叠

4. Pretokenization 与 Normalization：常被忽略，却会重写 Tokenizer 的搜索空间

许多“BPE tokenizer”之间的实际差别，不只在 merge 表，而在 merge 之前的切分规则。Pretokenizer 决定哪些字符永远不能被同一个 token 跨越，等于给词表学习施加 hard constraints。

设计点	影响	典型失败
Whitespace policy	是否把前导空格并入 token；代码缩进能否保留	把多空格折叠后破坏代码/表格结构
Regex boundaries	数字、标点、Unicode letter、newline 的分组	英语 regex 迁移到 CJK/代码后效率差
Unicode normalization	全角/兼容字符/组合字符是否合并	训练时 NFKC、线上 identity 导致 ID 分布漂移
Case normalization	大小写是否折叠	代码/专名信息损失
Byte fallback	极长尾字符、emoji、混合脚本的兜底	无 fallback 时出现 <unk> 或不可逆

Dagan 等对 code LLM 的系统 ablation 表明，词表大小、pre-tokenization regex 和 tokenizer 训练数据都会显著影响生成速度、有效上下文、内存和下游 coding 表现。[16]

Know-how Tokenizer 配置应和模型权重一样 versioned：normalizer、regex、merge rules、special tokens、ID mapping、decoder 规则缺一不可。只保存 vocab.json 而不保存 pretokenizer/normalizer，会制造不可复现的模型。

5. Vocabulary Size：为什么 32K 不再是默认答案

增大 V 的第一阶收益是减少 token 数；第一阶代价是增大 Embedding 与 LM Head。若 hidden size 为 d ，单个 embedding matrix 约有 $V \times d$ 参数；若输入 embedding 与输出 head 不 tied，则与 vocabulary 相关的参数接近 $2Vd$ 。

对固定原始文本，较大的 V 往往减少 L ；但输出 logits 每步要处理更大的 V 。于是总体效率来自“更少 steps”与“更贵的每 step”之间的 trade-off。

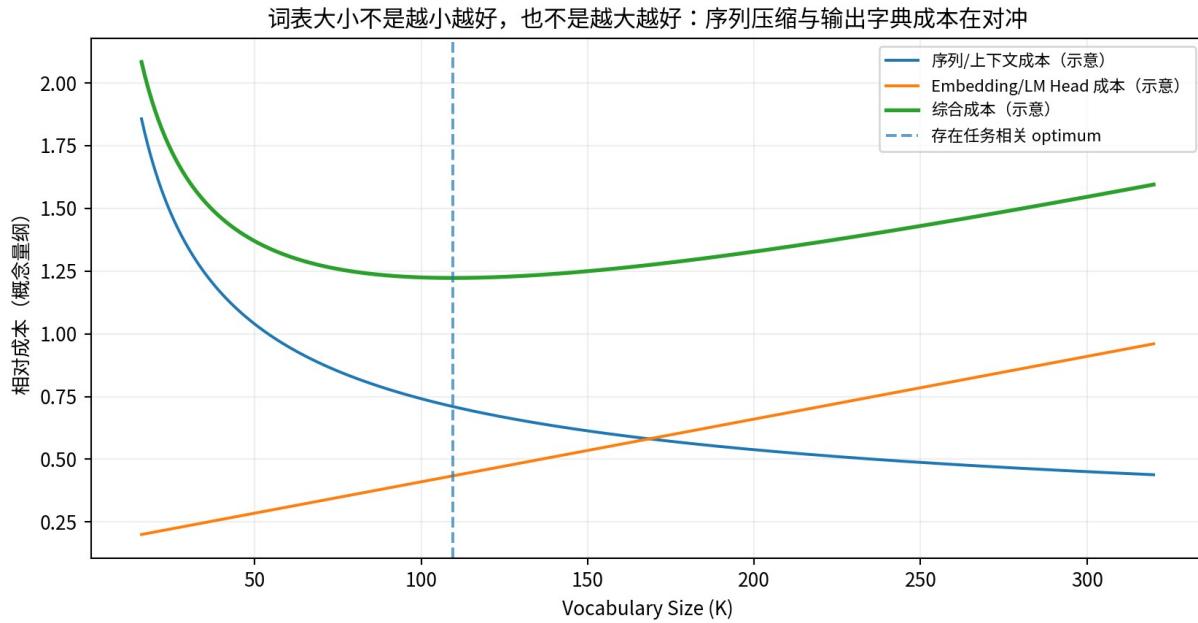


图 3 Vocabulary trade-off 概念图。曲线只表达机制，不是某个模型的实测最优值。

2024 的 Vocabulary Scaling Laws 通过 33M-3B 模型和最高 500B characters 的实验提出：compute-optimal vocabulary 会随 compute/model scale 增长；作者估计 Llama2-70B 的最优词表至少约 216K，而其实际词表更小。该结论应视为特定实验族的 scaling-law 预测，而非普适常数。[13]

同年的 24 个 2.6B 模型对照实验也显示，多语言 tokenizer 需要比英语 tokenizer 更大的词表才能避免高 tokenization cost；在其设置中，五种高频欧洲语言的 tokenizer 约需 3 倍词表规模。[12]

设计原则 Vocab Size 必须与 Model Size、Domain Mix、Language Mix、Context Budget、是否 tied embeddings 共同优化。把 32K/50K 当行业标准，是继承历史工程选择，而不是定律。

6. Tokenizer 的训练数据本身就是一份 Data Mixture

模型预训练数据和 tokenizer 训练数据是两个相关但不应完全混同的 corpus。Tokenizer 通常只需要从大语料中抽样一小部分来学频率结构；问题在于这个 sample 的 mixture 决定了哪些字符、词根、代码片段和语言值得占用稀缺 vocab slots。

Tokenizer corpus 决策	为什么重要	推荐做法
Language weights	自然比例会让英语吞噬 merge slots	对目标语言做 temperature / quota；单独报告 fertility
Domain weights	Web 高频字符串会压制 code/math token	为 code、math、JSON、Markdown 保留覆盖
Dedup	重复模板会把 merge frequency 放大	先做 representative dedup 或至少模板降权
Quality filtering	垃圾/乱码会占用词表	过滤控制字符、base64、追踪参数等
Time slice	新术语和 Unicode/emoji 分布会变化	与部署期 corpus 定期复评
Sample size	太小导致 merge ranking 方差大	做多个 seed/sample 的稳定性检查

Hayase 等 2024 反过来利用 BPE merge list 推断 tokenizer 训练数据 mixture，说明 merge 排序确实携带训练数据的语言、代码和来源比例信息。[17]

Know-why Tokenizer 可以被看成“训练语料频率的压缩指纹”。因此 tokenizer corpus 不只是工具输入，也是数据治理资产，应该记录来源、mix、过滤版本与训练 seed。

7. 多语言 Token Tax：有效上下文和算力可以因语言而系统性不公平

如果同样一句语义在语言 A 中需要 20 tokens、语言 B 需要 60 tokens，那么 B 在固定 token window 下可容纳的信息更少，同时生成相同长度内容需要更多 autoregressive steps。

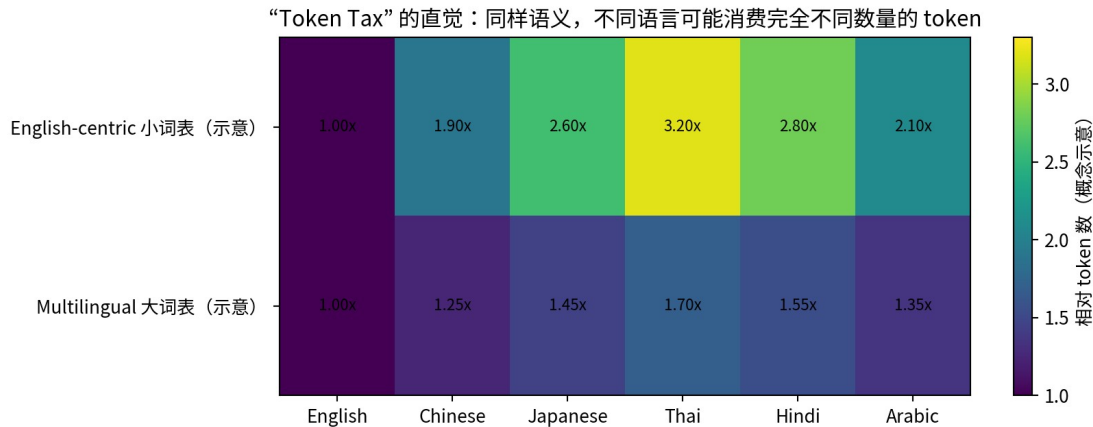


图 4 多语言 Token Tax 概念示意。数值为机制示意，不是特定模型实测。

Rust 等在 9 种语言的控制研究中发现：在相同预训练数据下，专门的单语言 tokenizer 本身就能显著影响下游表现；当语言在多语言词表中得到充分表示时，与单语模型的性能差距会明显缩小。[10]

Tokenizer Choice 2024 进一步发现，英语中心 tokenizer 用于多语言 LLM 会造成严重性能下降，并在其实验设置中引入最高约 68% 的额外训练成本；同时 fertility 和 parity 并不总能预测最终 downstream performance。[12]

2026 的 TokLens 对 24 个 tokenizer、15 种语言进行评估：GPT-2 在日语上相对英语产生极端高 fertility，而较新的 Qwen2.5/Gemma-2 把跨语言差距明显压低；其结果也表明 intrinsic metrics 对多语言性能有预测力，但不能替代真实 model ablation。[20]

多语言指标	定义/直觉	局限
Fertility	tokens / word	“word”对CJK/泰语定义不稳定
Chars per token	字符 / token	不同 Unicode script 字符信息量不同
Bytes per token	UTF-8 bytes / token	可跨 script 比较，但与人类语言单位弱
Parity	相同/平行语义的 token 长度比	依赖高质量平行语料
Single-token retention	常见词/术语成为单 token 的比例	无法直接预测组合泛化
Downstream delta	换 tokenizer 后真实模型性能变化	最可靠但成本最高

8. Code、Math、Numbers：Tokenizer 会不会破坏结构化符号？

自然语言 Tokenizer 的压缩目标未必适合 code/math。代码拥有大量空格、换行、缩进、运算符、snake_case、CamelCase、路径、十六进制和长 identifier；数学拥有数字、符号、LaTeX 与公式结构。

Slice	应测试的 tokenizer 行为	Why
Indentation	1/2/4/8 spaces、tab、newline 是否高效稳定	Python 等语言语义依赖缩进
Identifiers	snake_case / CamelCase / Unicode identifier	过度合并会降低组合性，过度拆分会拉长序列
Operators	=, !=, >, <, **, <= 等	高频运算符适合稳定 token
Numbers	逐位 / 2-3 位 chunk / 任意长数字	影响算术归纳与长度泛化
JSON/XML	quotes, braces, colon, tags	Agent/tool 数据高度依赖结构 token
Paths/URLs	/, ., :, ?, =, percent encoding	Web/code 语料频繁出现且易被错误 regex 切碎

Dagan 等在代码预训练/域适配中发现，专门优化 code tokenizer 可以提高生成速度和有效上下文，并在足够长的后续训练条件下带来性能收益；其结论直接支持“Tokenizer 是 domain adaptation 的优化旋钮”。[16]

Gemma 2 官方技术报告给出一个有代表性的设计：沿用 Gemma/Gemini tokenizer，使用 SentencePiece、split digits、preserved whitespace 与 byte-level encodings，词表 256K。[15]

Know-how 不要只在 Wikipedia/普通 prose 上测压缩率。Tokenizer Eval Suite 至少应有：自然语言、多语言、code、math、JSON/tool call、Markdown/HTML、URLs、emoji/Unicode、长数字和异常空白。

9. Special Tokens：Tokenizer 同时也是模型的“协议层”

除了普通 mergeable tokens，现代模型还包含 BOS/EOS/PAD、document boundary、FIM、chat role、tool call、multimodal placeholder 等特殊 token。它们不是普通自然语言词，而是控制模型状态转换的协议符号。

类型	例子（抽象）	设计要求
Sequence	<BOS>, <EOS>, <PAD>	训练、packing、generation stopping 一致
Document	<DOC_END>	attention mask 与边界 semantics 对齐
Chat	<SYSTEM>, <USER>, <ASSISTANT>	模板必须和 SFT/推理完全一致
Code FIM	<FIM_PREFIX>/<FIM_SUFFIX>	预训练时必须见过对应 serialization
Tool	<CALL>/<RESULT>	避免与普通文本发生未转义碰撞
Multimodal	<IMAGE>/<AUDIO> placeholders	token ID 和外部 encoder projection 对齐

OpenAI tiktoken 对 special tokens 显式区分 allowed/disallowed token，并允许在已有 encoding 上添加特殊 token；这类机制反映了“mergeable vocabulary”和“协议 vocabulary”应分层管理。[5][6]

生产事故模式 训练数据使用一套 chat/template special IDs，线上推理使用另一套；或者普通文本能够无意触发 reserved token。Tokenizer protocol 必须加入端到端 round-trip 与 template golden tests。

10. 有效上下文、FLOPs、显存与延迟：为什么“128K tokens”不是统一的信息容量

对某个数据切片 s ，可定义 compression efficiency $c_s = \text{raw units} / \text{tokens}$ 。固定 context length L_{tok} 下，有效原文容量约为 $L_{\text{raw}} = L_{\text{tok}} \times c_s$ 。Tokenizer 效率越差，同样 128K context 能装的文本越少。

对于 full attention，如果同一段 raw text 因 tokenizer 变差而 token 数放大 r 倍，则 attention score 相关计算近似按 r^2 增长，而逐 token MLP/embedding 等部分近似按 r 增长；真实系统还受 FlashAttention、packing、GQA、长上下文稀疏策略等影响。

成本项	近似依赖	Tokenizer 影响
Embedding lookup	$O(Ld)$	V 主要影响参数表大小， L 影响访问次数
Self-attention	$O(L^2d)$ full attention	token inflation 对长序列尤其昂贵
MLP blocks	$O(Ld^2)$	token inflation 线性增加 layer compute
LM Head	$O(LVd)$	大 V 增加每 token 输出成本；少 token 可部分抵消
KV Cache inference	$O(Ld \times \text{layers})$	更碎 tokenizer 占用更多 cache 和 decode steps
API pricing	通常按 tokens	同样语义的用户成本直接依赖 tokenizer

Meta 在 Llama 3 发布时报告：128K 词表 tokenizer 相比 Llama 2 提高编码效率，基准中可减少最高约 15% tokens，并帮助 8B 模型在增加参数的情况下维持推理效率。[14]

2026 的 Token Tax 研究在非洲语言 benchmark 中也观察到高 fertility 与低准确率相关，并强调 token inflation 会带来显著训练成本差异；这类结论应结合具体 attention 架构解释，而不是机械套用单一倍率。[21]

11. 代表性模型：Tokenizer 设计正在从 32K 走向更大、更语言/领域友好的词表

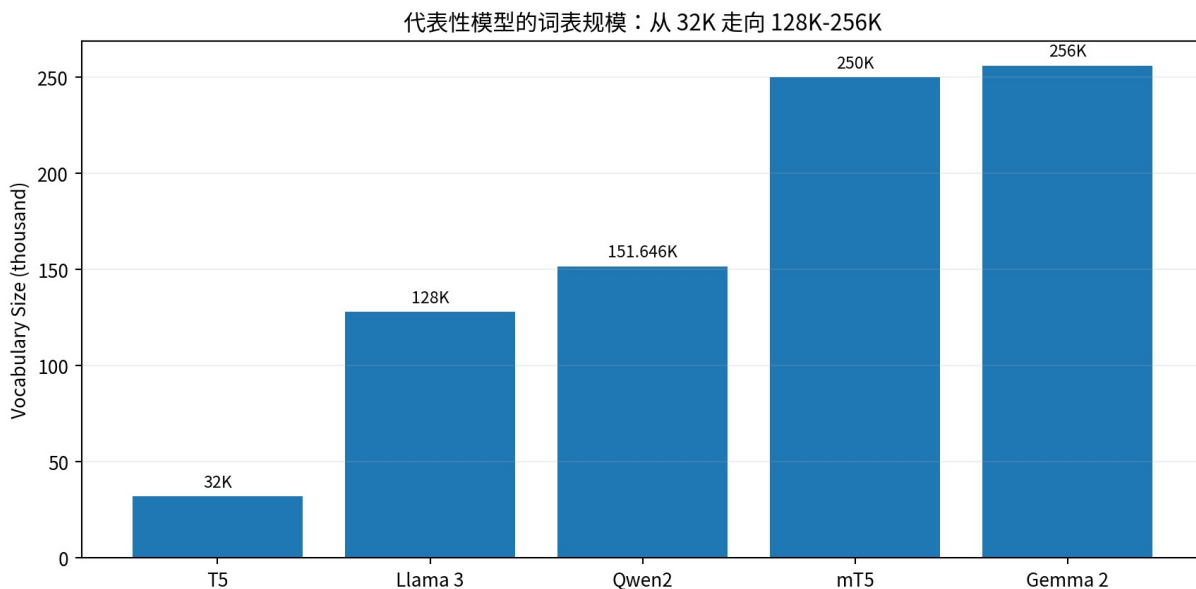


图 5 代表性公开模型的 vocabulary size（来源见本节引用）。

模型	Tokenizer / Vocab	关键设计信号
T5	SentencePiece, 约 32K	英语 C4 时代典型小词表；输入/输出共享 vocabulary
mT5	SentencePiece, 250K + extra IDs	101 语言，为多语言覆盖显著放大 vocab
Llama 3	TikToken-based, 128K	Meta 明确把更高 token efficiency 作为架构改进之一
Qwen2	byte-level BPE, 151,643 regular + 3 control	强调 multilingual compression efficiency
Gemma 2	SentencePiece, 256K	split digits + preserved whitespace + byte-level encodings

T5 官方代码使用 cc_all.32000 SentencePiece vocabulary；mT5/T5X 发布配置明确标注 mc4.250000.100extra。[7][8]

Qwen2 技术报告说明所有模型共享 151,643 regular tokens + 3 control tokens 的 byte-level BPE tokenizer，并将较好的 compression rate 与多语言能力联系起来。[18]

Gemma 2 报告使用 256K SentencePiece vocabulary；Llama 3 官方发布则公开 128K vocabulary 并报告更高 token efficiency。[15][14]

不要把 Vocab Size 当性能排名 更大词表通常改善特定数据分布的压缩，但模型质量还受 token semantics、训练语料、参数预算、head cost 与下游目标共同决定。上述表是设计趋势，不是“256K 一定优于 128K”。

12. Tokenizer Evaluation：Intrinsic Metrics 只能筛选，最终要靠 Proxy Pretraining

层级	指标	作用	不能回答什么
L0 正确性	round-trip、UNK rate、special token tests	保证实现不坏	不能说明能力
L1 压缩	tokens/word、chars/token、bytes/token	估算 context/成本	不能说明 token 是否“好学”
L2 公平性	language parity、tail-language fertility	发现 Token Tax	不能自动证明 downstream 提升
L3 结构性	code/math/number/whitespace slices	发现域特定失败	仍是 proxy
L4 系统	encode throughput、LM-head cost、KV/cache raw-text efficiency	判断端到端经济性	不能替代模型 loss/accuracy
L5 训练	small LM validation BPB + downstream eval	最接近真实效用	成本较高

Tokenizer Choice 2024 的关键警告是：fertility 与 parity 并不总能预测 downstream performance。因此 tokenizer 选择不能停在“哪个压缩率最好”。[12]

Vocabulary Scaling Laws 与 code tokenizer 研究都采用实际小模型训练来验证 vocabulary / regex / corpus 决策，说明 proxy pretraining 是 tokenizer design 的必要实验层。[13][16]

推荐决策函数 先用 intrinsic metrics 淘汰明显差方案，再用 100M-3B 级 proxy LM 在固定 raw-text / fixed-compute 条件做 A/B；最终大型训练前至少验证 loss/BPB、核心能力、训练 throughput 和 decode chars/sec。

13. 一个可复用的 Tokenizer Ablation 设计

变量	候选	保持不变
Algorithm	BPE vs Unigram	同 corpus sample、同 vocab size、同 normalizer
Vocab Size	32K/64K/128K/256K	同算法和 mixture
Pretokenizer	regex A/B/C	同 merge budget
Tokenizer data mix	natural vs balanced multilingual/domain	同 sample raw size
Byte policy	char coverage vs byte fallback	同 V
Digits	split digits on/off / grouped digits	同其他规则
Special reserve	minimal vs reserved slots	不让训练文本触发 reserved IDs

每个候选至少输出：①全局 tokens/byte；②按语言/域的 fertility；③ P50/P95 document token length；④ UNK/byte fallback 比例；⑤词表 token 的 frequency coverage；⑥ tokenizer throughput；⑦ proxy LM BPB；⑧ downstream slice；⑨固定 raw-text 下的 train FLOPs / tokens/sec。

实验原则 Tokenizer 方案的 A/B 必须区分两种预算：Fixed Tokens 与 Fixed Raw Text / Fixed Compute。只在 fixed tokens 下比较会奖励“更碎”的 tokenizer，因为它实际上让模型看到更少原始信息；只在 fixed raw text 下比较又会隐藏序列长度带来的计算成本。两种都需要。

14. 预训练后换 Tokenizer：为什么困难，以及 2024-2026 的可行路径

预训练后修改词表会打破 token ID -> embedding row -> LM head row 的一一对应。简单重新训练 tokenizer 后直接 resize，等于给模型更换“输入/输出字典”，原参数无法自然迁移。

方法	核心做法	风险/成本
Keep old tokenizer	只做 domain CPT	效率问题永久保留
Append new tokens	旧 IDs 不变，新 token 追加	需初始化新 embedding/head；V 增大
Tokenizer swap	训练全新 tokenizer	大量 token ID 不对应，恢复成本高
Continued BPE expansion	保留旧 merge，继续学新 merges	兼容性好，但只能在原词表基础上扩张
Embedding composition	新 token 用旧 subtokens embedding 组合初始化	需要后续 CPT 校准

Dagan 等指出，在足够长的后续训练（其 code 实验提到 50B+ tokens）下，专门化预训练模型的 tokenizer 可以带来速度与有效上下文收益。[16]

2026 In-Place Tokenizer Expansion 提出更保守的升级方式：继续原 BPE merges，保留既有 token 行，新 token 用其旧 subtoken embeddings 的均值初始化；随后先 embedding-only，再 full-model CPT。其 LFM2.5 实验对 Hindi/Vietnamese/Thai 显著减少 token 数，并估计每字符 decode 速度提升。该工作目前属于较新的预印本，需在更多模型上复现。[22]

同年一项 embedding initialization 系统研究进一步发现，轻量 CPT 的早期验证信号比“初始化时 loss/BPB”更可靠；subword-composition 类初始化在其 Hindi 扩词表实验中更优。该结果同样属于较新的预印本证据。[23]

Know-how 如果确定未来会扩语言/工具协议：预训练时就保留 unused/reserved IDs；若必须扩词表，优先保持旧 token IDs 与旧 merge 兼容，再做小规模 embedding adaptation + CPT，而不是一次性全量换字典。

15. Token-free / Dynamic Tokenization：固定词表是不是最终形态？

方向	核心想法	优势	代价
Byte-level LM	直接预测 bytes	完全开放词表、鲁棒、无 tokenizer drift	序列长
Character LM	直接预测 Unicode chars	语义边界较 byte 直观	字符集大、序列长
Subword regularization	训练时随机 segmentation	增强边界鲁棒性	推理通常仍固定 segmentation
Vocabulary curriculum	训练中逐步扩 vocabulary	让粒度随训练进展变化	实现和 checkpoint 兼容复杂
Over-tokenized input	输入 vocab 大、输出 vocab 解耦	扩大输入多粒度表示	不是传统共享 embedding 结构

ByT5 证明了 byte-to-byte 标准 Transformer 在若干任务上可与 token-level 模型竞争，并在噪声/拼写敏感任务上更鲁棒。[9]

2025 Vocabulary Curriculum 研究让词表在训练中按 entropy-guided 方式扩展；Over-Tokenized Transformer 则尝试把 input/output vocabulary 解耦并放大输入 vocab。两者都表明“静态 shared vocabulary”本身也是可被重新设计的架构假设，但目前主要仍属于研究前沿。[24][25]

16. Production Blueprint：Tokenizer 应该怎样进入真实预训练工程

阶段	输入	关键动作	输出资产
A. Requirements	语言/域/上下文/部署硬件	定义 token efficiency SLO 与协议需求	Tokenizer spec
B. Corpus sample	清洗后的 representative corpus	独立 mixture、dedup、Unicode audit	Versioned tokenizer corpus
C. Candidate train	algorithm/V/regex/normalizer	训练多个候选 + reserved IDs	Tokenizer candidates
D. Intrinsic eval	多语言/代码/math/异常切片	fertility、BPB proxy、UNK、round-trip	Scorecard
E. Proxy LM	固定架构小模型	fixed-token + fixed-raw-text A/B	Utility estimate
F. System eval	training/inference runtime	tok/s、chars/s、LM-head/comm、KV raw efficiency	Cost model
G. Freeze	winner	hash、ID map、special protocol、golden tests	Immutable tokenizer artifact
H. Monitor	新语言/新域/线上文本	drift、token tax、fallback rate	Expansion decision

推荐治理方式 Tokenizer 版本号应成为 checkpoint identity 的一部分，例如 model-v3 + tokenizer-v5。任何 tokenizer 变化都视为模型接口 breaking change，除非明确证明旧 token IDs 和 decoding semantics 完全兼容。

17. 典型失败模式：看起来“能 encode/decode”仍然可能是坏 Tokenizer

失败模式	表面现象	根因	诊断
Low-resource fragmentation	非英语回复慢、context 很快满	tokenizer corpus 英语占比过高	language fertility / parity

失败模式	表面现象	根因	诊断
Code whitespace explosion	代码 token 数异常高	regex/whitespace policy 不适配	code slice + indentation tests
Garbage vocab slots	词表含大量 URL tracking/base64 片段	tokenizer corpus 未清洗/去重	top tokens audit
Normalization drift	同一文本离线/线上 IDs 不同	normalizer 版本或 Unicode policy 不一致	golden round-trip tests
Special collision	普通文本触发控制 token	reserved token surface 设计不安全	fuzz special-token scanner
Vocab too large	参数/softmax/通信异常高	只优化压缩率	end-to-end fixed-compute eval
Vocab too small	长序列、KV/attention 成本高	沿用旧时代 32K 默认	vocab scaling ablation
Tokenizer swap collapse	CPT 初期 loss 爆炸	embedding/head ID 语义破坏	overlap + init + staged CPT

18. 最终 Know-Why / Know-How 沉淀

原则	Know-why	Know-how
1. Tokenizer 属于模型	它改变训练目标、L、V 和协议	和 architecture/data 一起做设计评审
2. 压缩不是唯一目标	高 chars/token 不保证 downstream 最优	intrinsic + proxy LM 双层评测
3. Vocab 要 scale	模型越大、语言越多，较大 V 往往更合理	至少做 3-4 个 vocab size IsoFLOPs A/B
4. Tokenizer corpus 要混合	merge rules 是数据频率的指纹	独立 version tokenizer mixture
5. 多语言要测 Token Tax	不同语言的 token 经济性不同	逐语言 fertility/bytes-per-token/context parity
6. Code/Math 要有专门切片	自然语言 regex 会破坏结构符号	测 whitespace/number/operator/JSON
7. Special tokens 是协议	ID 错位等于模型通信协议错位	reserved IDs + golden templates
8. 跨 tokenizer PPL 不可直接比	token 粒度不同	使用 BPB/BPC + downstream
9. Tokenizer 迁移要保持兼容	embedding/head 绑定 IDs	优先 append/continued merges + staged CPT
10. 大规模前必须 proxy	tokenizer 优劣最终是训练 utility	小模型 fixed-compute/fixed-raw-text ablation

一句话总结 Tokenizer 的真正工程目标不是“切得漂亮”，而是在目标语言与领域上，用有限 Vocab/Compute 把尽可能多、尽可能有用的原始信息压进每个 token，同时让这种离散化仍然利于模型学习、可逆、可扩展、可公平部署。

附录 A. Tokenizer Design Checklist

- 定义目标语言、domain、context、部署硬件与成本 SLO；不要先选 32K/128K 再找理由。
- Tokenizer corpus 与 pretraining corpus 分开 version，记录 mixture、sample、filter、dedup 和 seed。
- 至少比较 BPE/Unigram 或说明为何只选一种；BPE 时明确 char-level 还是 byte-level base。
- 记录 normalization、pretokenization regex、whitespace、digit、byte fallback 全部参数。
- 保证正常输入无 <unk>；特殊 token surface 与 raw text 做碰撞/fuzz 测试。
- 同时测试自然语言、多语言、code、math、JSON/tool、URLs、emoji、异常 whitespace。
- 报告 tokens/word、chars/token、bytes/token、P95 document length 和 per-language parity。
- 把 Embedding/LM Head 参数、softmax/通信、KV raw-text efficiency 纳入成本模型。
- 至少做 3 个 vocab sizes 的 proxy pretraining，并同时提供 fixed-token 与 fixed-compute/fixed-raw-text 结果。
- 跨 tokenizer 模型 loss 用 BPB/BPC 对齐；不要直接用 token PPL 排名。
- 冻结后保存 immutable model file / merges / vocab / special IDs / config hash / golden encode-decode cases。
- 部署后监控新语言、新 domain 的 fertility 和 fallback；达到阈值才触发 tokenizer expansion 评估。

附录 B. 参考资料与证据等级

证据等级：A = 同行评审论文或官方技术报告/官方实现；B = 较新的预印本/研究前沿，结论应等待更多复现。

[1] [A] Sennrich, Haddow, Birch. Neural Machine Translation of Rare Words with Subword Units. ACL 2016. [source](#)

[2] [A] Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. 2018. [source](#)

- [3] [A] Kudo, Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer. EMNLP 2018. [source](#)
- [4] [A] Google SentencePiece official repository and options documentation. [source](#)
- [5] [A] OpenAI tiktoken official repository: educational BPE implementation and tokenizer API. [source](#)
- [6] [A] OpenAI gpt-oss tokenizer: o200k-based encoding plus reserved/special tokens. [source](#)
- [7] [A] Google Research T5 official code: cc_all.32000 SentencePiece vocabulary. [source](#)
- [8] [A] Google Research T5X model docs: mT5 vocabulary mc4.250000.100extra. [source](#)
- [9] [A] Xue et al. ByT5: Towards a token-free future with pre-trained byte-to-byte models. 2021. [source](#)
- [10] [A] Rust et al. How Good is Your Tokenizer? ACL 2021. [source](#)
- [11] [A] Bostrom, Durrett. Byte Pair Encoding is Suboptimal for Language Model Pretraining. 2020. [source](#)
- [12] [A] Ali et al. Tokenizer Choice For LLM Training: Negligible or Crucial? Findings NAACL 2024. [source](#)
- [13] [B] Tao et al. Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies. 2024. [source](#)
- [14] [A] Meta. Introducing Meta Llama 3. 128K vocabulary; up to 15% fewer tokens vs Llama 2 in reported benchmarks. [source](#)
- [15] [A] Gemma Team. Gemma 2: Improving Open Language Models at a Practical Size. 2024. [source](#)
- [16] [B] Dagan, Synnaeve, Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. 2024. [source](#)
- [17] [B] Hayase et al. Data Mixture Inference: What do BPE Tokenizers Reveal about their Training Data? 2024. [source](#)
- [18] [A] Qwen Team. Qwen2 Technical Report. 151,643 regular + 3 control tokens; byte-level BPE. 2024. [source](#)
- [19] [A] Xue et al. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. NAACL 2021. [source](#)
- [20] [A] Chiu. TokLens: A Multilingual Lens on Tokenizer Quality for LLMs. ACL SRW 2026. [source](#)
- [21] [A] Lundin et al. The Token Tax: Systematic Bias in Multilingual Tokenization. AfricaNLP 2026. [source](#)
- [22] [B] Smith et al. In-Place Tokenizer Expansion for Pre-trained LLMs. 2026. [source](#)
- [23] [B] Joshi et al. Beyond Initialization Loss: Token Embedding Initialization for LLM Vocabulary Extension. 2026. [source](#)
- [24] [B] Yu. Scaling LLM Pre-training with Vocabulary Curriculum. 2025. [source](#)
- [25] [B] Huang et al. Over-Tokenized Transformer: Vocabulary is Generally Worth Scaling. 2025. [source](#)

研究说明：本文对 2025-2026 的 tokenizer expansion、vocabulary curriculum、over-tokenized 等工作明确按“前沿/待复现”处理；对于模型官方 tokenizer 规格，优先采用模型团队技术报告或官方实现。